The Priority-Aware Warehouse Dispatcher

Context

You are designing the control logic for an automated warehouse. Packages arrive on a **sequential conveyor belt**, and your task is to assign them to one of **K** available loading docks.

Constraints

1. **Fixed Order:** You must process packages in the exact order they arrive on the belt. You cannot skip a package or reorder them.
2. **Dock Capacity (M):** Every loading dock has the same maximum weight capacity, **M**. The sum of the weights of all packages assigned to a single dock cannot exceed **M**.
3. **Priority Rule:** Each package has a priority level (an integer). To ensure efficient loading, packages assigned to the **same dock** must be in **non-decreasing order of priority**.
    - *Example:* If the last package sent to Dock 1 had Priority 5, you can only send a new package to Dock 1 if its priority is ≥ 5.
    - If a package violates this rule, you **must** move to the next available loading dock.
4. **Dock Usage:** Once you move from Dock *i* to Dock *i+1*, Dock *i* is considered "sealed" and sent for delivery. You cannot go back to a previous dock.

The Goal

Given an array of package **weights**, an array of package **priorities**, and the number of available docks **K**, find the **minimum possible capacity M** required to successfully dispatch all packages.

Example

Input:

- weights = [5, 3, 7, 2, 4]
- priorities = [2, 4, 3, 5, 6]
- K = 3

Logic Walkthrough for M = 10:

- **Dock 1:**
    - Pkg 1 (Wt: 5, Pr: 2) -> Added.
    - Pkg 2 (Wt: 3, Pr: 4) -> Added (Total Wt: 8, Pr: 4 ≥ 2).

    - Pkg 3 (Wt: 7, Pr: 3) -> **Fail** (Priority 3 < 4). *Move to Dock 2.*
- **Dock 2:**
    - Pkg 3 (Wt: 7, Pr: 3) -> Added.
    - Pkg 4 (Wt: 2, Pr: 5) -> Added (Total Wt: 9, Pr: 5 ≥ 3).
    - Pkg 5 (Wt: 4, Pr: 6) -> **Fail** (Total Wt: 13 > 10). *Move to Dock 3.*
- **Dock 3:**
    - Pkg 5 (Wt: 4, Pr: 6) -> Added.
- **Result:** All packages fit in 3 docks. M=10 is a valid (though perhaps not minimal) capacity.